

Smart Contract Assessment

Moonwell

HALBORN

Smart Contract Assessment - Moonwell

Prepared by:  HALBORN

Last Updated 12/05/2025

Date of Engagement: November 21st, 2025 - November 25th, 2025

Summary

100% ⓘ OF ALL REPORTED FINDINGS HAVE BEEN ADDRESSED

ALL FINDINGS	CRITICAL	HIGH	MEDIUM	LOW	INFORMATIONAL
14	0	1	0	7	6

TABLE OF CONTENTS

1. Introduction	
2. Assessment summary	
2.1 Caveats	
3. Test approach and methodology	
4. Risk methodology	
5. Scope	
6. Assessment summary & findings overview	
7. Findings & Tech Details	
7.1 Fee logic allows free oev updates	
7.2 Improper round decrement at phase change	
7.3 No revert on failed reserve addition	
7.4 Underflow risk with high-decimal tokens	
7.5 Non-standard ERC20 tokens may break transfers	
7.6 Unsafe symbol based feed lookup	
7.7 Single step ownership transfer process	
7.8 Lack of whitelist removal functionality	
7.9 Redundant cached round updates allowed	

7.10 Missing input validation

7.11 Reentrancy risk in sensitive functions

7.12 Inconsistent round validation logic

7.13 Missing event

7.14 Use of revert strings instead of custom errors

1. INTRODUCTION

Moonwell engaged **Halborn** to perform a security assessment of their smart contracts from November 21st, 2025 to November 25th, 2025. The assessment scope was limited to the smart contracts provided to Halborn. Commit hashes and additional details are available in the Scope section of this report.

The **Moonwell** codebase in scope consists of smart contracts designed to wrap Chainlink price feeds with OEV gating, integrate with Morpho Blue for liquidations, and manage protocol fee redemption.

2. ASSESSMENT SUMMARY

Halborn was allocated 3 days for this engagement and assigned 2 full-time security engineers to conduct a comprehensive review of the smart contracts within scope. The engineers are experts in blockchain and smart contract security, with advanced skills in penetration testing and smart contract exploitation, as well as extensive knowledge of multiple blockchain protocols.

The objectives of this assessment are to:

- Identify potential security vulnerabilities within the smart contracts.
- Verify that the smart contract functionality operates as intended.

In summary, **Halborn** identified several areas for improvement to reduce the likelihood and impact of security risks, which were partially addressed by the **Moonwell team**. The primary recommendations were:

- Redesign the fee calculation to require a minimum fee based on actual gas used or a fixed minimum, and ensure it cannot be bypassed by sending transactions with zero priority fee.
- Add an owner-controlled emergencyMode flag that requires payment for all price updates and disables fallback to unpaid rounds during phase transitions.
- Check that `mToken._addReserves(amount)` returns `0` and revert otherwise, as done in `ChainlinkFeed0EVWrapper`.

2.1 CAVEATS

It's important to note that the finding *HAL-01: Fee logic allows free OEV updates*, classified with **HIGH** severity, was not remediated, as the **Moonwell** team states that the issue pertains to a deprecated file.

3. TEST APPROACH AND METHODOLOGY

Halborn conducted a combination of manual code review and automated security testing to balance efficiency, timeliness, practicality, and accuracy within the scope of this assessment. While manual testing is crucial for identifying flaws in logic, processes, and implementation, automated testing enhances coverage of smart contracts and quickly detects deviations from established security best practices.

The following phases and associated tools were employed throughout the term of the assessment:

- Research into the platform's architecture, purpose and use.
- Manual code review and walkthrough of smart contracts to identify any logical issues.
- Comprehensive assessment of the safety and usage of critical Solidity variables and functions within scope that could lead to arithmetic-related vulnerabilities.
- Local testing using custom scripts (Foundry).
- Fork testing against main networks (Foundry).
- Static security analysis of scoped contracts, and imported functions (Slither).

4. RISK METHODOLOGY

Every vulnerability and issue observed by Halborn is ranked based on **two sets of Metrics** and a **Severity Coefficient**. This system is inspired by the industry standard Common Vulnerability Scoring System.

The two **Metric sets** are: **Exploitability** and **Impact**. **Exploitability** captures the ease and technical means by which vulnerabilities can be exploited and **Impact** describes the consequences of a successful exploit.

The **Severity Coefficients** is designed to further refine the accuracy of the ranking with two factors: **Reversibility** and **Scope**. These capture the impact of the vulnerability on the environment as well as the number of users and smart contracts affected.

The final score is a value between 0-10 rounded up to 1 decimal place and 10 corresponding to the highest security risk. This provides an objective and accurate rating of the severity of security vulnerabilities in smart contracts.

The system is designed to assist in identifying and prioritizing vulnerabilities based on their level of risk to address the most critical issues in a timely manner.

4.1 EXPLOITABILITY

ATTACK ORIGIN (AO):

Captures whether the attack requires compromising a specific account.

ATTACK COST (AC):

Captures the cost of exploiting the vulnerability incurred by the attacker relative to sending a single transaction on the relevant blockchain. Includes but is not limited to financial and computational cost.

ATTACK COMPLEXITY (AX):

Describes the conditions beyond the attacker’s control that must exist in order to exploit the vulnerability. Includes but is not limited to macro situation, available third-party liquidity and regulatory challenges.

METRICS:

EXPLOITABILITY METRIC (M_E)	METRIC VALUE	NUMERICAL VALUE
Attack Origin (AO)	Arbitrary (AO:A) Specific (AO:S)	1 0.2

EXPLOITABILITY METRIC (M_E)	METRIC VALUE	NUMERICAL VALUE
Attack Cost (AC)	Low (AC:L)	1
	Medium (AC:M)	0.67
	High (AC:H)	0.33
Attack Complexity (AX)	Low (AX:L)	1
	Medium (AX:M)	0.67
	High (AX:H)	0.33

Exploitability E is calculated using the following formula:

$$E = \prod m_e$$

4.2 IMPACT

CONFIDENTIALITY (C):

Measures the impact to the confidentiality of the information resources managed by the contract due to a successfully exploited vulnerability. Confidentiality refers to limiting access to authorized users only.

INTEGRITY (I):

Measures the impact to integrity of a successfully exploited vulnerability. Integrity refers to the trustworthiness and veracity of data stored and/or processed on-chain. Integrity impact directly affecting Deposit or Yield records is excluded.

AVAILABILITY (A):

Measures the impact to the availability of the impacted component resulting from a successfully exploited vulnerability. This metric refers to smart contract features and functionality, not state. Availability impact directly affecting Deposit or Yield is excluded.

DEPOSIT (D):

Measures the impact to the deposits made to the contract by either users or owners.

YIELD (Y):

Measures the impact to the yield generated by the contract for either users or owners.

METRICS:

Impact Metric (M_I)	Metric Value	Numerical Value
Confidentiality (C)	None (C:N)	0
	Low (C:L)	0.25
	Medium (C:M)	0.5
	High (C:H)	0.75
	Critical (C:C)	1
Integrity (I)	None (I:N)	0
	Low (I:L)	0.25
	Medium (I:M)	0.5
	High (I:H)	0.75
	Critical (I:C)	1
Availability (A)	None (A:N)	0
	Low (A:L)	0.25
	Medium (A:M)	0.5
	High (A:H)	0.75
	Critical (A:C)	1
Deposit (D)	None (D:N)	0
	Low (D:L)	0.25
	Medium (D:M)	0.5
	High (D:H)	0.75
	Critical (D:C)	1
Yield (Y)	None (Y:N)	0
	Low (Y:L)	0.25
	Medium (Y:M)	0.5
	High (Y:H)	0.75
	Critical (Y:C)	1

Impact I is calculated using the following formula:

$$I = max(m_I) + \frac{\sum m_I - max(m_I)}{4}$$

4.3 SEVERITY COEFFICIENT

REVERSIBILITY (R):

Describes the share of the exploited vulnerability effects that can be reversed. For upgradeable contracts, assume the contract private key is available.

SCOPE (S):

Captures whether a vulnerability in one vulnerable contract impacts resources in other contracts.

METRICS:

SEVERITY COEFFICIENT (<i>C</i>)	COEFFICIENT VALUE	NUMERICAL VALUE
Reversibility (<i>r</i>)	None (R:N)	1
	Partial (R:P)	0.5
	Full (R:F)	0.25
Scope (<i>s</i>)	Changed (S:C)	1.25
	Unchanged (S:U)	1

Severity Coefficient *C* is obtained by the following product:

$$C = rs$$

The Vulnerability Severity Score *S* is obtained by:

$$S = min(10, EIC * 10)$$

The score is rounded up to 1 decimal places.

SEVERITY	SCORE VALUE RANGE
Critical	9 - 10
High	7 - 8.9
Medium	4.5 - 6.9
Low	2 - 4.4

SEVERITY	SCORE VALUE RANGE
Informational	0 - 1.9

5. SCOPE

REPOSITORY
(a) Repository: moonwell-contracts-v2 (b) Assessed Commit ID: e688eb3 (c) Items in scope: - src/oracles/ChainlinkFeedOEVWrapper.sol - src/oracles/ChainlinkOEVMorphoWrapper.sol - src/OEVProtocolFeeRedeemer.sol Out-of-Scope: Third party dependencies and economic attacks.
REMEDIATION COMMIT ID:
96fca48 - d9dc61d 53bfbcf - c355929 5ffd9c3
Out-of-Scope: New features/implementations after the remediation commit IDs.

6. ASSESSMENT SUMMARY & FINDINGS OVERVIEW

CRITICAL	HIGH	MEDIUM	LOW	INFORMATIONAL
0	1	0	7	6
SECURITY ANALYSIS		RISK LEVEL		REMEDIATION DATE
FEE LOGIC ALLOWS FREE OEV UPDATES		HIGH		RISK ACCEPTED - 12/03/2025

SECURITY ANALYSIS	RISK LEVEL	REMEDATION DATE
IMPROPER ROUND DECREMENT AT PHASE CHANGE	LOW	RISK ACCEPTED - 12/03/2025
NO REVERT ON FAILED RESERVE ADDITION	LOW	SOLVED - 12/02/2025
UNDERFLOW RISK WITH HIGH-DECIMAL TOKENS	LOW	SOLVED - 12/02/2025
NON-STANDARD ERC20 TOKENS MAY BREAK TRANSFERS	LOW	RISK ACCEPTED - 12/03/2025
UNSAFE SYMBOL BASED FEED LOOKUP	LOW	RISK ACCEPTED - 12/03/2025
SINGLE STEP OWNERSHIP TRANSFER PROCESS	LOW	RISK ACCEPTED - 12/03/2025
LACK OF WHITELIST REMOVAL FUNCTIONALITY	LOW	SOLVED - 12/02/2025
REDUNDANT CACHED ROUND UPDATES ALLOWED	INFORMATIONAL	ACKNOWLEDGED - 12/03/2025
MISSING INPUT VALIDATION	INFORMATIONAL	SOLVED - 12/02/2025
REENTRANCY RISK IN SENSITIVE FUNCTIONS	INFORMATIONAL	SOLVED - 12/02/2025

SECURITY ANALYSIS	RISK LEVEL	REMEDIATION DATE
INCONSISTENT ROUND VALIDATION LOGIC	INFORMATIONAL	ACKNOWLEDGED - 12/03/2025
MISSING EVENT	INFORMATIONAL	SOLVED - 12/02/2025
USE OF REVERT STRINGS INSTEAD OF CUSTOM ERRORS	INFORMATIONAL	ACKNOWLEDGED - 12/03/2025

7. FINDINGS & TECH DETAILS

7.1 FEE LOGIC ALLOWS FREE OEV UPDATES

// HIGH

Description

The `updatePriceEarly()` function in the `ChainlinkFeed0EVWrapper` contract is intended to require a fee for early price updates, calculated as:

```
203 | require(  
204 |     msg.value >= (tx.gasprice - block.basefee) * uint256(feeMultiplier),  
205 |     "Chainlink0EVWrapper: Insufficient tax"  
206 | );
```

 Copy Code

However, on EIP-1559 compatible networks, transactions can be sent with a zero priority fee (`max_priority_fee_per_gas = 0`), making `tx.gasprice - block.basefee` equal to zero.

According to the EIP-1559 specification:

"Transactions specify the maximum fee per gas they are willing to give to miners to incentivize them to include their transaction (aka: priority fee). ... Senders will always pay the base fee per gas of the block their transaction was included in, and they will pay the priority fee per gas set in the transaction, as long as the combined amount of the two fees doesn't exceed the transaction's maximum fee per gas."

 Copy Code

```
# priority fee is capped because the base fee is filled first  
priority_fee_per_gas = min(transaction.max_priority_fee_per_gas, transaction.max_fee_per_gas - bloc
```

This allows any account to update the cached round and claim OEV revenue without paying any fee, defeating the protocol's intended revenue model.

In practice, this vulnerability is highly exploitable by MEV searchers and liquidators, who routinely use `maxPriorityFeePerGas = 0` in private bundles submitted to block builders. These actors can guarantee transaction inclusion without paying a priority fee, as builders are incentivized by MEV and block rewards, not by the priority fee. As a result, the protocol will lose the vast majority of its intended OEV revenue, as nearly all profitable liquidations could bypass the fee mechanism.

Proof of Concept

In the following simple scenario, the fee value can be bypassed with the same logic as the current `ChainlinkFeed0EVWrapper` contract:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.19;

import {Test, console2} from "@forge-std/Test.sol";

contract OEVPoCTester is Test {
    uint public feeMultiplier = 100;

    function setUp() public {
        vm.createSelectFork("https://base.llamarpc.com");
    }

    function test_FeesCanBeZero() external payable {
        vm.txGasPrice(block.basefee);
        console2.log("tx.gasprice:", tx.gasprice);
        console2.log("block.basefee:", block.basefee);
        console2.log("feeMultiplier:", feeMultiplier);
        uint256 fee = (tx.gasprice - block.basefee) * uint256(feeMultiplier);
        console2.log("calculated fee:", fee);
        require(msg.value >= fee, "Should not revert");
    }
}
```

Ran 1 test for test/unit/Halborn.t.sol:OEVPoCTester

[PASS] test_FeesCanBeZero() (gas: 11646)

Logs:

tx.gasprice: 13269519

block.basefee: 13269519

feeMultiplier: 100

calculated fee: 0

Test passed: No fee required for zero priority fee tx

Suite result: **ok**. 1 passed; 0 failed; 0 skipped; finished in 2.12s (159.86ms CPU time)

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:H/Y:N (7.5)

Recommendation

Redesign the fee calculation to require a minimum fee based on actual gas used or a fixed minimum, and ensure it cannot be bypassed by sending transactions with zero priority fee. Additionally, consider scaling the fee to reflect the real cost of the update and the value of the OEV opportunity.

Remediation Comment

RISK ACCEPTED: The **Moonwell** team made a business decision to accept the risk of this finding and not alter the contracts, as this issue pertains to a deprecated version of the **ChainlinkFeed0EVWrapper** contract.

References

<https://eips.ethereum.org/EIPS/eip-1559>

7.2 IMPROPER ROUND DECREMENT AT PHASE CHANGE

// LOW

Description

The `latestRoundData()` function in both `Chainlink0EVMorphoWrapper` and `ChainlinkFeed0EVWrapper` attempts to return a previous round's data if the latest round has not been "paid for" (i.e., OEV fee not collected or liquidation not triggered). However, the function decrements the round ID linearly without considering Chainlink's phase encoding (where round IDs are packed as $(\text{phaseId} \ll 64) \mid \text{aggregatorRoundId}$).

- In `ChainlinkFeed0EVWrapper.latestRoundData()`:

 Copy Code

```
125 // If the current round is not too old and hasn't been paid for,
126 // attempt to find the most recent valid round by checking previous rounds
127 for (uint256 i = 0; i < maxDecrements && --startRoundId > 0; i++) {
128     try originalFeed.getRoundData(uint80(startRoundId)) returns (
129         uint80 r,
130         int256 a,
131         uint256 s,
132         uint256 u,
133         uint80 ar
134     ) {
135         _validateRoundData(r, a, u, ar);
136
137         roundId = r;
138         answer = a;
139         startedAt = s;
140         updatedAt = u;
141         answeredInRound = ar;
142         return (roundId, answer, startedAt, updatedAt, answeredInRound);
143     } catch {}
144 }
```

- In `Chainlink0EVMorphoWrapper.latestRoundData()`:

 Copy Code

```
239 for (uint256 i = 0; i < maxDecrements && currentRoundId > 0; i++) {
240     try priceFeed.getRoundData(uint80(currentRoundId)) returns (
241         uint80 r,
242         int256 a,
243         uint256 s,
244         uint256 u,
245         uint80 ar
246     ) {
247         // previous round data found, update the round data
248         roundId = r;
249         answer = a;
250         startedAt = s;
251         updatedAt = u;
252         answeredInRound = ar;
253         break;
254     } catch {
255         // previous round data not found, continue to the next decrement
256         currentRoundId--;
257     }
258 }
```

```
}  
}
```

When a phase change occurs, decrementing the round ID can result in a non-existent round, causing the loop to fail and the function to fall back to returning the latest (unpaid) round. This allows to bypass the intended OEV gating and access the latest price without paying the required fee.

BVSS

AO:A/AC:L/AX:M/R:N/S:U/C:N/A:M/I:N/D:N/Y:N (3.4)

Recommendation

Given that Chainlink phase changes are extremely rare and typically announced in advance, it is recommended to implement an operational safeguard (rather than complex phase-aware round navigation). Specifically:

- Add an `emergencyMode` flag controlled by the contract owner (e.g., `setEmergencyMode()`).
- When `emergencyMode` is enabled, modify `latestRoundData()` to require payment for all price updates if the latest round has not been paid for, preventing fallback to unpaid rounds during a phase transition.
- Operationally, monitor Chainlink for announced phase upgrades, enable `emergencyMode` before the phase change, and disable it after the transition period (e.g., 24 hours).

For additional reference, see [Chainlink's documentation on Getting Historical Data](#).

Remediation Comment

RISK ACCEPTED: The **Moonwell team** made a business decision to accept the risk of this finding and not alter the contracts, stating that:

| Chainlink phase changes are extremely rare.

References

<https://docs.chain.link/data-feeds/historical-data#historical-rounds>

7.3 NO REVERT ON FAILED RESERVE ADDITION

// LOW

Description

The `_addReservesToMToken()` function in `0EVProtocolFeeRedeemer` calls `mToken._addReserves(amount)` but does not check its return value. According to the `MErc20Interface`, `_addReserves` returns an error code (0 for success).

If the call fails (nonzero return), the contract will emit a `ReservesAddedFrom0EV` event and continue as if reserves were added. Funds remain in the contract, and reserves are not incremented, creating accounting/monitoring inconsistencies.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:L/D:N/Y:N (2.5)

Recommendation

Check that `mToken._addReserves(amount)` returns 0 and revert otherwise, as done in `ChainlinkFeed0EVWrapper`.

Remediation Comment

SOLVED: The `Moonwell` team solved this finding in the specified commit by following the mentioned recommendation.

Remediation Hash

<https://github.com/moonwell-fi/moonwell-contracts-v2/commit/96fca4859df5295f9db1ef6829a48cdedf4cf212>

7.4 UNDERFLOW RISK WITH HIGH-DECIMAL TOKENS

// LOW

Description

In `ChainlinkOVMorphoWrapper`, functions such as `_getLoanTokenPrice` and `_getCollateralTokenPrice` assume that both the Chainlink feed and the ERC20 token have decimals ≤ 18 .

If either has decimals > 18 , the calculation `uint256(18) - uint256(decimals)` will underflow and revert, potentially breaking liquidations or price queries.

BVSS

AO:S/AC:L/AX:L/R:N/S:U/C:N/A:C/I:N/D:N/Y:N (2.0)

Recommendation

Add explicit checks to ensure that feed and token decimals do not exceed 18, or handle higher decimals safely.

Remediation Comment

SOLVED: The `Moonwell` team solved this finding in the specified commit by following the mentioned recommendation.

Remediation Hash

<https://github.com/moonwell-fi/moonwell-contracts-v2/commit/d9dc61d67199f6d66200d420f9019f8f733beff0>

7.5 NON-STANDARD ERC20 TOKENS MAY BREAK TRANSFERS

// LOW

Description

Both `Chainlink0EVMorphoWrapper` and `0EVProtocolFeeRedeemer` use the standard `approve` and `transferFrom` patterns for ERC20 tokens. Some tokens (e.g., USDT) require allowance to be set to zero before updating to a new value, and fee-on-transfer tokens may not transfer the expected amount. This can cause liquidations or reserve additions to fail or behave unexpectedly.

BVSS

AO:S/AC:L/AX:L/R:N/S:U/C:N/A:C/I:N/D:N/Y:N (2.0)

Recommendation

Consider using OpenZeppelin's `SafeERC20` library for approvals and transfers.

Remediation Comment

RISK ACCEPTED: The `Moonwell` team made a business decision to accept the risk of this finding and not alter the contracts, stating that:

| Tokens [are] managed by governance.

7.6 UNSAFE SYMBOL BASED FEED LOOKUP

// LOW

Description

The `_getLoanTokenPrice()` function in `ChainlinkOVMorphoWrapper` resolves the price feed by calling `chainlinkOracle.getFeed(loanToken.symbol())`.

This approach is fragile because ERC20 symbols are not unique, can be spoofed, and may change. An attacker could deploy a token with a colliding symbol, causing the contract to resolve to an incorrect or malicious feed.

BVSS

AO:S/AC:L/AX:L/R:N/S:U/C:N/A:C/I:N/D:N/Y:N (2.0)

Recommendation

Use token addresses as keys for feed resolution instead of symbols to ensure uniqueness and prevent spoofing.

Remediation Comment

RISK ACCEPTED: The `Moonwell` team made a business decision to accept the risk of this finding and not alter the contracts, stating that:

 | Symbols [are] managed by governance.

7.7 SINGLE STEP OWNERSHIP TRANSFER PROCESS

// LOW

Description

The `OEVProtocolFeeRedeemer` and the `ChainlinkFeed0EVWrapper` contracts inherit the `Ownable` contract implementation from OpenZeppelin's library and is used to restrict access to certain functions to the contract owner. Similarly, the `Chainlink0EVMorphoWrapper` inherits from `OwnableUpgradeable`. The `Ownable` and `OwnableUpgradeable` patterns allow the contract owner to transfer ownership to another address using the `transferOwnership()` function. However, the `transferOwnership()` function does not include a two-step process to transfer ownership.

Regarding this, it is crucial that the address to which ownership is transferred is verified to be active and willing to assume ownership responsibilities. Otherwise, the contract could be locked in a situation where it is no longer possible to make administrative changes to it.

Additionally, the `renounceOwnership()` function in these modules allow renouncing to the owner permission. Renouncing ownership before transferring it would result in the contract having no owner, eliminating the ability to call privileged functions.

BVSS

AO:S/AC:L/AX:L/R:N/S:U/C:N/A:C/I:N/D:N/Y:N (2.0)

Recommendation

Consider using OpenZeppelin's `Ownable2Step` or (`Ownable2StepUpgradeable`) contracts over the `Ownable` implementation. `Ownable2Step` provides a two-step ownership transfer process, which adds an extra layer of security to prevent accidental ownership transfers.

Additionally, it is recommended that the owner cannot call the `renounceOwnership()` function to avoid losing ownership of the contract.

Remediation Comment

RISK ACCEPTED: The **Moonwell** team made a business decision to accept the risk of this finding and not alter the contracts, stating that:

| Ownership [is] not being transferred beyond governance.

7.8 LACK OF WHITELIST REMOVAL FUNCTIONALITY

// LOW

Description

The `OEVProtocolFeeRedeemer` contract provides an `whitelistMarket(address _market)` function that allows the contract owner to add a market to the `whitelistedMarkets` mapping, enabling it to interact with reserve related functions. However, there is no corresponding function to remove a market from the whitelist.

This means that once a market is whitelisted, it cannot be revoked, even if the market becomes deprecated, compromised, or otherwise undesirable. This could lead to operational or security risks if a previously whitelisted market should no longer be trusted.

BVSS

AO:S/AC:L/AX:L/R:N/S:U/C:N/A:N/I:C/D:N/Y:N (2.0)

Recommendation

Implement a function to allow the owner to remove markets from the whitelist.

Remediation Comment

SOLVED: The `Moonwell` team solved this finding in the specified commit by following the mentioned recommendation.

Remediation Hash

<https://github.com/moonwell-fi/moonwell-contracts-v2/commit/96fca4859df5295f9db1ef6829a48cdedf4cf212>

7.9 REDUNDANT CACHED ROUND UPDATES ALLOWED

// INFORMATIONAL

Description

The `updatePriceEarlyAndLiquidate()` function in `Chainlink0EVMorphoWrapper` updates the `cachedRoundId` to the latest Chainlink round and performs a liquidation. However, it does not check whether the latest Chainlink round ID is greater than the current `cachedRoundId`. This allows an account to repeatedly call the function and update `cachedRoundId` to the same value if the Chainlink feed is stalled (i.e., no new rounds are published).

The absence of this check does not introduce a new economic or security risk, but it may lead to unnecessary state changes and event emissions.

BVSS

AO:A/AC:L/AX:M/R:N/S:U/C:N/A:N/I:L/D:N/Y:N (1.7)

Recommendation

Optionally enforce `require(roundId > cachedRoundId)` to avoid redundant writes.

Remediation Comment

ACKNOWLEDGED: The `Moonwell` team made a business decision to acknowledge this finding and not alter the contracts, stating that:

Round [is] updated on liquidations.

7.10 MISSING INPUT VALIDATION

// INFORMATIONAL

Description

In `ChainlinkOEVWrapper.initializeV2`, the `_morphoBlue` parameter is not checked for the zero address, which could lead to misconfiguration and failed liquidations.

Similarly, `OEVProtocolFeeRedeemer` does not validate that `MOONWELL_WETH` is non-zero in its constructor, and `ChainlinkFeedOEVWrapper` does not validate constructor addresses.

Failing to validate inputs before assigning them to state variables can lead to unexpected system behavior or even complete failure.

BVSS

AO:S/AC:L/AX:L/R:N/S:U/C:N/A:M/I:N/D:N/Y:N (1.0)

Recommendation

Add zero-address checks for all critical contract addresses during initialization and construction.

Remediation Comment

SOLVED: The `Moonwell` team solved this finding in the specified commit by following the mentioned recommendation.

Remediation Hash

<https://github.com/moonwell-fi/moonwell-contracts-v2/commit/53bfbcf82d2c70bfcddcafad0710970d2f886049>

7.11 REENTRANCY RISK IN SENSITIVE FUNCTIONS

// INFORMATIONAL

Description

The `updatePriceEarlyAndLiquidate()` function in `Chainlink0EVMorphoWrapper`, the `updatePriceEarly()` function in `ChainlinkFeed0EVWrapper`, and the `redeemAndAddReserves()` and reserve-related functions in `0EVProtocolFeeRedeemer` all perform external calls without a reentrancy guard. These functions update contract state and interact with external contracts or tokens, which could be malicious or non-standard.

While the protocol expects only standard tokens and trusted contracts, this lack of protection increases the risk of unexpected behavior if a non-standard or malicious contract is introduced.

BVSS

AO:A/AC:L/AX:H/R:N/S:U/C:N/A:N/I:L/D:L/Y:N (1.0)

Recommendation

Add a reentrancy guard (e.g., a `nonReentrant` modifier) to all functions that perform external calls and update state, and all reserve-related functions in `0EVProtocolFeeRedeemer`.

Remediation Comment

SOLVED: The `Moonwell` team solved this finding in the specified commit by following the mentioned recommendation.

Remediation Hash

<https://github.com/moonwell-fi/moonwell-contracts-v2/commit/c355929a369eac87b21889d0e5d9da3571687f0d>

7.12 INCONSISTENT ROUND VALIDATION LOGIC

// INFORMATIONAL

Description

In `ChainlinkFeed0EVWrapper`, the `getRoundData()` function does not call `_validateRoundData()`, unlike `latestRoundData()`. This means callers of `getRoundData()` do not benefit from the same data freshness and integrity checks.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:N (0.0)

Recommendation

Apply consistent validation logic in both `getRoundData` and `latestRoundData`.

Remediation Comment

ACKNOWLEDGED: The `Moonwell` team made a business decision to acknowledge this finding and not alter the contracts, as this issue pertains to a deprecated version of the `ChainlinkFeed0EVWrapper` contract.

7.13 MISSING EVENT

// INFORMATIONAL

Description

The `whitelistMarket()` function in the `OEVProtocolFeeRedeemer` contract changes the contract state by modifying a core state variable but without emitting any event.

The absence of events may hamper effective state tracking in off-chain monitoring systems.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:N (0.0)

Recommendation

Emit an event for the aforementioned function to facilitate off-chain monitoring of the system.

Remediation Comment

SOLVED: The `Moonwell` team solved this finding in the specified commit by following the mentioned recommendation.

Remediation Hash

<https://github.com/moonwell-fi/moonwell-contracts-v2/commit/5ffd9c33a590990c8feb16739d065e144381bcc1>

7.14 USE OF REVERT STRINGS INSTEAD OF CUSTOM ERRORS

// INFORMATIONAL

Description

Throughout the files in scope, there are several instances where revert strings are used over custom errors.

In Solidity development, replacing hard-coded revert message strings with the `Error()` syntax is an optimization strategy that can significantly reduce gas costs. Hard-coded strings, stored on the blockchain, increase the size and cost of deploying and executing contracts.

The `Error()` syntax allows for the definition of reusable, parameterized custom errors, leading to a more efficient use of storage and reduced gas consumption. This approach not only optimizes gas usage during deployment and interaction with the contract but also enhances code maintainability and readability by providing clearer, context-specific error information.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:N (0.0)

Recommendation

Consider replacing all revert strings with custom errors. For example:

```
error ConditionNotMet(); if (!condition) revert ConditionNotMet();
```

 Copy Code

or starting from Solidity `0.8.27` :

```
require(condition, ConditionNotMet());
```

 Copy Code

Remediation Comment

ACKNOWLEDGED: The `Moonwell` team made a business decision to acknowledge this finding and not alter the contracts, stating that:

Using custom errors would deviate from rest of the codebase.

Halborn strongly recommends conducting a follow-up assessment of the project either within six months or immediately following any material changes to the codebase, whichever comes first. This approach is crucial for maintaining the

project's integrity and addressing potential vulnerabilities introduced by code modifications.